

# Software Requirements Specification (SRS)

## Student Registration Management System

### 1. Introduction

#### 1.1 Purpose

The Student Registration Management System (SRMS) is a web-based application designed to automate the academic registration process within a university.

The system enables administrators to create academic structures, lecturers to manage courses, and students to register for semesters and courses while maintaining complete academic records.

The application will be implemented using:

- Java 21
  - Spring Boot
  - Spring MVC
  - Spring Data JPA (Hibernate)
  - Spring Security
  - PostgreSQL
  - Maven
  - REST API Architecture
- 

### 2. Problem Statement

Most universities still experience problems such as:

- Manual student registration
- Duplicate registrations
- Course allocation errors
- Poor semester management

- Difficulty generating reports
- Poor tracking of student academic history

The proposed system automates these activities while ensuring data consistency and integrity.

---

## 3. Objectives

The system shall:

- Register students
  - Manage academic departments
  - Manage faculties
  - Manage programmes
  - Manage semesters
  - Manage teachers
  - Manage courses
  - Register students every semester
  - Register students into courses
  - Record student results
  - Produce academic reports
- 

## 4. Users

There are four categories of users.

### 4.1 Administrator

Responsible for system management.

Can:

- Login
- Create departments
- Create faculties
- Create programmes
- Create semesters
- Register teachers

- Register students
  - Create courses
  - Assign teachers
  - Generate reports
- 

## 4.2 Registrar

Responsible for student registration.

Can:

- Register students
  - Register semesters
  - Register courses
  - View student history
- 

## 4.3 Lecturer

Responsible for teaching.

Can:

- View assigned courses
  - View registered students
  - Record marks
  - Update results
- 

## 4.4 Student

Can:

- Login
- View profile
- Register semester
- Register courses
- View results
- View transcript

---

## 5. Functional Requirements

---

### FR1 Student Management

The system shall allow administrators to manage student records.

#### Student Information

- Registration Number
- First Name
- Last Name
- Date of Birth
- Email
- Phone Number
- Gender
- Status

#### Operations

- Add Student
- Update Student
- Delete Student
- Search Student
- View Student Profile

#### Business Rules

Registration Number must be unique.

---

### FR2 Academic Unit Management

Academic Units represent the university hierarchy.

The system supports three types:

- Faculty
- Programme
- Department

Each Academic Unit contains:

- Code
- Name
- Type

An academic unit may contain child academic units.

Example

Engineering Faculty

↓

Computer Science Programme

↓

Software Engineering Department

This is represented using a parent-child relationship.

---

## FR3 Semester Management

A semester represents an academic period.

Attributes

- Semester ID
- Name
- Start Date
- End Date
- Status

Operations

Create Semester

Update Semester

Activate Semester

Close Semester

Only one semester can be active.

---

## FR4 Teacher Management

Each teacher contains

- Code
- Names
- Qualification

Qualification values

- Master
- PhD
- Professor

Operations

Create Teacher

Update Teacher

Delete Teacher

Assign Courses

---

## FR5 Course Definition

A course template.

Contains

- Course Code
- Course Name
- Description

Examples

INSY8311

Database Development

Credits

Prerequisites

---

## FR6 Course Offering

A course offered during a semester.

Contains

Course Definition

Semester

Department

Tutor

Assistant Tutor

Maximum Capacity

Status

This allows one course to be offered multiple semesters.

---

## FR7 Student Semester Registration

Students register every semester.

Contains

Student

Semester

Registration Date

Status

Business Rules

A student cannot register twice in the same semester.

---

## **FR8 Student Course Registration**

After semester registration, students choose courses.

Contains

Student Registration

Course

Credits

Results

Business Rules

Student must already be registered for the semester.

Cannot register the same course twice.

Maximum credit limit must be enforced.

Course capacity must not be exceeded.

---

## **FR9 Result Management**

Lecturers enter results.

Each record contains



Student

Course

Marks

Grade

GPA

Remarks

Results can only be entered after semester completion.

---

## FR10 Authentication

Users login using

Username

Password

Roles

ADMIN

REGISTRAR

LECTURER

STUDENT

Security

Password encryption

JWT Authentication

Role-based authorization

---

## 6. UML Explanation

---

### Student

Represents every university student.

Attributes

Registration Number

First Name

Date of Birth

Relationships

One Student

↓

Many Semester Registrations

---

### Academic Unit

Represents organizational structures.

Faculty

Programme

Department

Uses a recursive relationship.

Parent

↓

Child

Example

Faculty



Programme



Department

---

## Student Registration

Represents semester registration.

Links

Student



Semester

This is the student's registration record for one semester.

---

## Student Course

Represents course enrollment.

Links

Student Registration



Course

Stores

Credits

Results

---

## Semester

Represents academic periods.

Contains

Start Date

End Date

One semester contains many registrations.

---

## Course Definition

Stores permanent course information.

Examples

Programming II

Database Systems

Software Engineering

Never changes.

---

## Course

Represents an offered course.

Contains

Course Definition

Department

Tutor

Assistant Tutor

Semester

One course belongs to one department.

---

## Teacher

Represents lecturers.

Attributes

Code

Names

Qualification

Relationships

Tutor

Assistant Tutor

A teacher may teach multiple courses.

---

## 7. Database Design

Main Tables

Student

AcademicUnit

Semester

Teacher

CourseDefinition

Course

StudentRegistration

StudentCourse

User

Role

Permission

AuditLog

---

## 8. Relationships

Student



StudentRegistration

(One to Many)

StudentRegistration



StudentCourse

(One to Many)

Semester



StudentRegistration

(One to Many)

CourseDefinition



Course

(One to Many)

Teacher



Course

(One to Many)

AcademicUnit



Course

(One to Many)

AcademicUnit



AcademicUnit

(Self Reference)

---

## 9. Spring Boot Architecture

The implementation shall follow a layered architecture.

Controller Layer



Service Layer



Repository Layer



Database

### Controller Layer

Exposes REST APIs.

Responsibilities

Receive HTTP Requests

Validate Inputs

Return HTTP Responses

---

## **Service Layer**

Contains business rules.

Responsibilities

Validation

Transactions

Registration Logic

Business Processing

---

## **Repository Layer**

Uses Spring Data JPA.

Responsibilities

CRUD

Queries

Pagination

Filtering

---

## **Entity Layer**

Represents database tables.

Includes

Relationships



Constraints

Validation

---

## DTO Layer

Transfers data between client and server.

Avoids exposing entities.

---

## Mapper Layer

Converts

DTO

↓

Entity

Entity

↓

DTO

---

## Security Layer

Spring Security

JWT

Role Authorization

Password Encryption

Authentication

---

## Exception Handling

Global Exception Handler

Custom Exceptions

Validation Errors

Database Errors

---

## Logging

Audit

Student Registration

Login

Course Registration

Result Entry

---

## 10. API Modules

Student API

Academic Unit API

Semester API

Teacher API

Course Definition API

Course API

Registration API

Student Course API

Authentication API

## 11. Validation Rules

Student

Registration Number required

Unique Registration Number

Email format

Valid birth date

Semester

End Date must be after Start Date

One active semester

Course

Unique Code

Credits positive

Teacher

Unique Code

Qualification required

Student Registration

No duplicate semester registration

Student Course

No duplicate course registration

Credit limit enforced

---

# 12. Reporting Requirements

The system shall generate the reports specified in the assignment:

## Report 1

List of Students per Semester

Columns

Student ID

Student Name

Semester

Registration Date

---

## Report 2

List of Students per Department and Semester

Columns

Department

Semester

Student

Registration Date

---

## Report 3

List of Students per Course and Semester

Columns

Course

Semester

Student

Marks

---

## Report 4

List of Courses per Department and Semester

Columns

Department

Semester

Course

Tutor

Assistant Tutor

---

## Report 5

List of Courses per Student

Columns

Student

Course

Credits

Semester

Tutor

Assistant Tutor

This satisfies the note in the diagram requiring both the **Tutor** and **Assistant Tutor** to appear in course listings.

---

# 13. Non-Functional Requirements

## Performance

- Support at least 1,000 concurrent users.
- Typical API response time should be under 2 seconds under normal load.

## Security

- JWT-based authentication.
- Passwords stored using BCrypt.
- Role-Based Access Control (RBAC).
- HTTPS in production.
- Audit logging for sensitive operations.

## Reliability

- Transaction management for all registration operations.
- Automatic rollback on failures.
- Regular database backup strategy.

## Scalability

- Stateless REST APIs.
- Layered architecture to support horizontal scaling.
- Database indexing for frequently queried fields.

## Maintainability

- Modular package structure.
- Separation of concerns.
- API documentation using OpenAPI/Swagger.
- Unit, integration, and end-to-end test support.

## Usability

- Responsive web interface.
  - Clear validation messages.
  - Consistent navigation and error handling.
-

# 14. Software Testing Requirements

Since this is for a **Software Testing** course, every functional requirement should be verifiable through test cases.

## Unit Testing

- Service-layer business logic (registration rules, credit limits, duplicate prevention).
- Validation methods.
- Utility and mapper components.

## Integration Testing

- Controller-to-service-to-database workflows.
- JPA repository operations.
- Authentication and authorization.

## API Testing

- CRUD operations for all modules.
- Authentication endpoints.
- Report generation endpoints.
- Validation and error responses.

## System Testing

- Complete registration workflow from student creation through semester registration, course enrollment, and result publication.
- Role-based access control verification.
- End-to-end report generation.

## Performance Testing

- Concurrent student registrations.
- Report generation with large datasets.
- Database query response times.

## Security Testing

- Authentication and authorization.
- Password encryption.
- Protection against SQL injection, XSS, and CSRF (where applicable).
- Access control for administrator, registrar, lecturer, and student roles.

## Acceptance Testing

- Verify that all five required reports are generated correctly.
  - Confirm business rules (no duplicate registrations, active semester enforcement, credit limits, tutor/assistant tutor visibility, etc.).
  - Validate that the system meets all functional and non-functional requirements.
- 

## Expected Deliverables

A complete implementation should include:

1. Spring Boot REST API following a layered architecture.
2. PostgreSQL (or MySQL) database with normalized schema.
3. Authentication and role-based authorization using Spring Security and JWT.
4. CRUD modules for Students, Academic Units, Teachers, Semesters, Course Definitions, Course Offerings, and Registrations.
5. Comprehensive input validation and centralized exception handling.
6. Reporting module implementing all five required reports.
7. API documentation (OpenAPI/Swagger).
8. Unit, integration, API, system, performance, security, and acceptance tests with documented test cases and results.